



TECHNICAL PRESENTATION

Introduction to Data Structures

Course: Data Structure

**DONE BY
SRIVATSAN L**

What is a Data Structure?

- 🗄️ A data structure is a specialized format for organizing, processing, retrieving, and storing data effectively in a computer's memory.
- 🔗 Different types of data structures are suited to different kinds of applications, and some are highly specialized to specific tasks.
- 🧠 Understanding data structures provides a crucial means to manage large amounts of data efficiently.
- 🕒 This organization is the absolute cornerstone of writing performant and scalable algorithms.



Classification of Data Structures

Linear Data Structures

Data elements are arranged sequentially or linearly, where each element is attached to its previous and next adjacent elements in memory.

- **Arrays:** Contiguous memory blocks.
- **Linked Lists:** Connected individual nodes.
- **Stacks:** Last-In-First-Out (LIFO).
- **Queues:** First-In-First-Out (FIFO).

Non-Linear Data Structures

Data elements are not placed sequentially. They are arranged in a hierarchical manner, with elements connected to multiple other elements.

- **Trees:** Parent-child relationships (e.g., Binary Search Trees).
- **Graphs:** Interconnected nodes via edges.
- **Hash Tables:** Key-value pair associations mapping indexes to data.

Linked Lists: Dynamic Memory

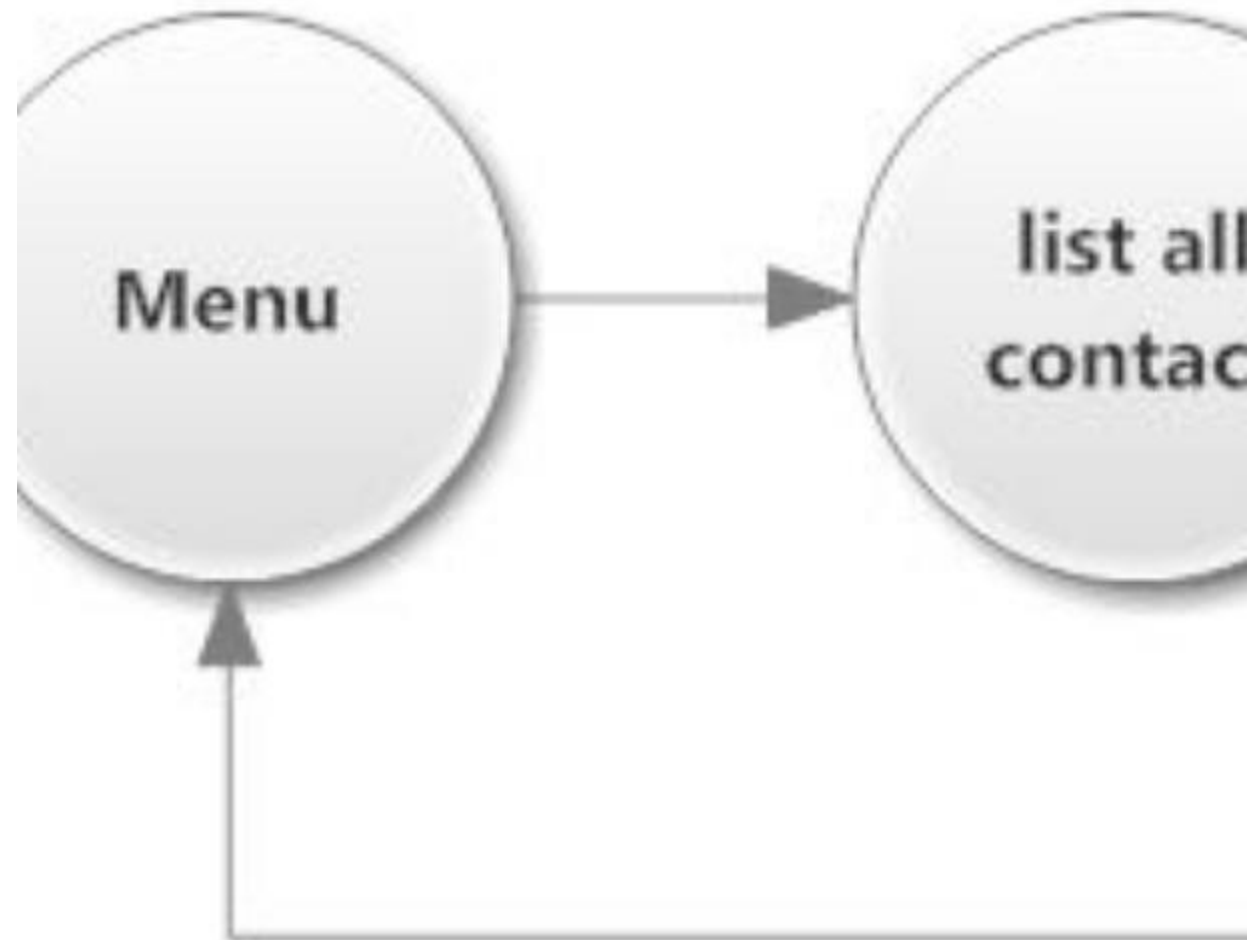
Overcoming Fixed Memory

Linked Lists consist of nodes where each node contains data and a pointer (or reference) to the next node in the sequence.

Unlike arrays, memory does not need to be contiguous. This dynamic structure allows for efficient insertion and deletion of elements without shifting large blocks of data.

Variations include:

- Singly Linked Lists (one-way pointers)
- Doubly Linked Lists (two-way pointers)
- Circular Linked Lists (looping structure)



Stacks vs. Queues

Stacks (LIFO)

Operates on a **Last-In, First-Out** principle, much like a stack of plates. The last element added is the first one to be removed.

Key Operations: Push (add), Pop (remove).

Use Cases:

- Undo mechanisms in text editors.
- Browser history navigation.
- Function call stack (handling recursion).

Queues (FIFO)

Operates on a **First-In, First-Out** principle, similar to a line at a ticket counter. The first element added is the first one out.

Key Operations: Enqueue (add), Dequeue (remove).

Use Cases:

- Task scheduling in operating systems.
- Printer spooling operations.
- Breadth-First Search (BFS) in graphs.

